

Document number: P3111R0.

Date: 2024-5-16.

Authors: Gonzalo Brito Gadeschi, Simon Cooksey, Daniel Lustig.

Reply to: Gonzalo Brito Gadeschi <gonzalob_at_nvidia.com (<http://nvidia.com>)>.

Audience: SG1, SG6.

Table of Contents

- Atomic Reduction Operations
 - Introduction
 - Motivation
 - Hardware Exposure
 - Performance
 - Design
 - Alternative: optimizations for `fetch_<key>`
 - Forward progress
 - Generalized Atomic Reduction Operations
 - Memory Ordering
 - Formalization
 - Wording
 - Unsequenced support
 - Forward progress
 - No acquire sequences support
 - Atomic Reduction Operation APIs

Atomic Reduction Operations

Atomic Reduction Operations are atomic read-modify-write (RMW) operations (like `fetch_add`) that do not "fetch" the old value and are not reads from the Memory Model perspective. This enables implementations to leverage hardware acceleration available in modern CPU and GPU architectures.

Furthermore, we propose to allow atomic memory operations that aren't reads in unsequenced execution, and to extend atomic arithmetic reduction operations for floating-point types with operations that assume floating-point arithmetic is associative.

Introduction

Concurrent algorithms performing atomic RMW operations that discard the old fetched value are very common in high-performance computing, e.g., finite-element matrix assembly, data analytics (e.g. building histograms), etc.

Consider the following parallel algorithm to build a histogram (full implementation (<https://clang.godbolt.org/z/zscrhEPYj>)):

```
// Example 0: Histogram.
span<unsigned> data;

array<atomic<unsigned>, N> buckets;
constexpr T bucket_sz = numeric_limits<T>::max() / (T)N;
unsigned nthreads = thread::hardware_concurrency();
for_each_n(execution::par_unseq, views::iota(0).begin(), nthreads,
    [&](int thread) {
        unsigned data_per_thread = data.size() / nthreads;
        T* data_thread = data.data() + data_per_thread * thread;
        for (auto e : span<T>(data_thread, data_per_thread))
            buckets[e / bucket_sz].fetch_add(1, memory_order_relaxed);
    });
```

This program has two main issues:

- **Correctness** (undefined behavior): The program should have used the `par` execution policy to avoid undefined behavior since the atomic operation is not:
 - Potentially concurrent and therefore exhibits a data-race in unsequenced contexts ([intro.execution]).
 - Vectorization-safe [algorithms.parallel.defns#5] (<https://eel.is/c++draft/algorithms.parallel.defns#5>) since it is specified to synchronize with other function invocations.
- **Performance**: Sophisticated compiler analysis required to optimize the above program for scalable hardware architectures with atomic reduction operations.

Atomic reduction operations address both shortcomings:

Before (compiler-explorer https://clang.godbolt.org/z/xq8efq1WK)	After
<pre> #include <algorithm> #include <atomic> #include <execution> using namespace std; using execution::par_unseq; int main() { size_t N = 10000; vector<int> v(N, 0); atomic<int> atom = 0; for_each_n(par_unseq, v.begin(), N, [&](auto& e) { // UB+SLOW: atom.fetch_add(e); }); return atom.load(); } </pre>	<pre> #include <algorithm> #include <atomic> #include <execution> using namespace std; using execution::par_unseq; int main() { size_t N = 10000; vector<int> v(N, 0); atomic<int> atom = 0; for_each_n(par_unseq, v.begin(), N, [&](auto& e) { // OK+FAST atom.add(e); }); return atom.load(); } </pre>

This new operation can then be used in the Histogram Example (example 0), to replace the `fetch_add` with just `add`.

Motivation

Hardware Exposure

The following ISAs provide Atomic Reduction Operation:

Architecture	Instructions
PTX	<code>red</code> (https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#parallel-synchronization-and-communication-instructions-red).
ARM	<code>LDADD RZ</code> (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/LDADD--LDADDA--LDADDAL--LDADDL--Atomic-add-on-word-or-doubleword-in-memory-?lang=en), <code>STADD</code> (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/STADD--STADDL--Atomic-add-on-word-or-doubleword-in-memory--without-return--an-alias-of-LDADD--LDADDA--LDADDAL--LDADDL-), <code>SWP RZ</code> (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/SWP--SWPA--SWPAL--SWPL--Swap-word-or-doubleword-in-memory-?lang=en), <code>CAS RZ</code> (https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/CAS--CASA--CASAL--CASL--Compare-and-Swap-word-or-doubleword-in-memory-?lang=en).
x86-64	Remote Atomic Operations (RAO) (https://cdrdv2-public.intel.com/671368/architecture-instruction-set-extensions-programming-reference.pdf): <code>AADD</code> , <code>AAND</code> , <code>AOR</code> , <code>AXOR</code> .
RISC-V	None (note: AMOs are always loads and stores).
PP64LE	None.

Some of these instructions lack a destination operand (`red` (<https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#parallel-synchronization-and-communication-instructions-red>), `STADD` (<https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/STADD--STADDL--Atomic-add-on-word-or-doubleword-in-memory--without-return--an-alias-of-LDADD--LDADDA--LDADDAL--LDADDL->), `AADD`). Others change semantics if the destination register used discards the result (Arm's `LDADD RZ` (<https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/LDADD--LDADDA--LDADDAL--LDADDL--Atomic-add-on-word-or-doubleword-in-memory-?lang=en>), `SWP RZ` (<https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/SWP--SWPA--SWPAL--SWPL--Swap-word-or-doubleword-in-memory-?lang=en>), `CAS RZ` (<https://developer.arm.com/documentation/ddi0602/2022-06/Base-Instructions/CAS--CASA--CASAL--CASL--Compare-and-Swap-word-or-doubleword-in-memory-?lang=en>)).

All ISAs provide the same semantics: these are not loads from the point-of-view of the Memory Model, and therefore do not participate in acquire sequences, but they do participate in release sequences:

- PTX Specification: `red` is "not a read operation (<https://docs.nvidia.com/cuda/parallel-thread-execution/index.html#operation-types>)".
- Arm ARM: "where the destination register is WZR or XZR, are not regarded as doing a read for the purpose of a DMB LD barrier (<https://developer.arm.com/documentation/ddi0487/latest>)".

- x86-64: "since they do not load data from memory into the processor." (<https://cdrdv2-public.intel.com/671368/architecture-instruction-set-extensions-programming-reference.pdf>)

These architectures provide both "relaxed" and "release" orderings for the reductions (e.g. `red.relaxed / red.release` , `STADD / STADDL`).

Performance

On hardware architectures that implement these as far atomics, the exposed latency of Atomic Reduction Operations may be as low as half that of " `fetch_<key>` " operations.

Example: on an NVIDIA Hopper H100 GPU, replacing `atomic.fetch_add` with `atomic.add` on the Histogram Example (Example 0) improves throughput by 1.2x.

Design

For each `atomic fetch_{0P}` in the `atomic<T>` and `atomic_ref<T>` class templates and their specializations, we introduce new `{0P}` member functions that return `void` .

These can be conservatively implemented on top of `fetch_{0P}` APIs.

Alternative: optimizations for `fetch_<key>`

Attempting to improve application performance by implementing compiler-optimizations to leverage Atomic Reduction Operations from `fetch_<key>` APIs has become a rite of passage for compiler engineers, e.g., [GCC#509632](https://gcc.gnu.org/pipermail/gcc-patches/2018-October/509632.html) (<https://gcc.gnu.org/pipermail/gcc-patches/2018-October/509632.html>), [LLVM#68428](https://github.com/llvm/llvm-project/issues/68428) (<https://github.com/llvm/llvm-project/issues/68428>), [LLVM#72747](https://github.com/llvm/llvm-project/pull/72747) (<https://github.com/llvm/llvm-project/pull/72747>), ... Unfortunately, "simple" optimization strategies break backward compatibility in the following litmus tests (among others).

Litmus Test 0: from Will Deacon (<https://gcc.gnu.org/pipermail/gcc-patches/2018-October/509632.html>). Performing the optimization to replace the introduces the `y == 2 && r0 == 1 && r1 == 0` outcome:

```

void thread0(atomic_int* y,atomic_int* x) {
    atomic_store_explicit(x,1,memory_order_relaxed);
    atomic_thread_fence(memory_order_release);
    atomic_store_explicit(y,1,memory_order_relaxed);
}

void thread1(atomic_int* y,atomic_int* x) {
    atomic_fetch_add_explicit(y,1,memory_order_relaxed);
    atomic_thread_fence(memory_order_acquire);
    int r0 = atomic_load_explicit(x,memory_order_relaxed);
}

void thread2(atomic_int* y) {
    int r1 = atomic_load_explicit(y,memory_order_relaxed);
}

```

Litmus Test 1: from Luke Geeson (<https://github.com/llvm/llvm-project/issues/68428#issue-1930595855>). Performing the optimization of replacing the exchange with a store introduces the `r0 == 0 && y == 2` outcome:

```

void thread0(atomic_int* y,atomic_int* x) {
    atomic_store_explicit(x,1,memory_order_relaxed);
    atomic_thread_fence(memory_order_release);
    atomic_store_explicit(y,1,memory_order_relaxed);
}

void thread1(atomic_int* y,atomic_int* x) {
    atomic_exchange_explicit(y,2,memory_order_release);
    atomic_thread_fence(memory_order_acquire);
    int r0 = atomic_load_explicit(x,memory_order_relaxed);
}

```

In some architectures, Atomic Reduction Operations can write to memory pages that are not readable, and need a reliable programming model that does not depend on compiler-optimizations for functionality.

Forward progress

Currently, all atomic memory operations are vectorization-unsafe (<https://eel.is/c++draft/algorithms.parallel.defns#5>) and therefore not allowed in element access functions of parallel algorithms when the `unseq` or `par_unseq` execution policies are used (see [\[algorithms.parallel.exec.5\]](#) (<https://eel.is/c++draft/algorithms.parallel#exec-5>) and [\[algorithms.parallel.exec.7\]](#) (<https://eel.is/c++draft/algorithms.parallel#exec-7>)). Atomic memory operations that "read" (e.g. `load`, `fetch_<key>`, `compare_exchange`, `exchange`, ...) enable building synchronization edges that block, which within `unseq` / `par_unseq` leads to dead-

locks. N4070 (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2014/n4070.html>) solved this by tightening the wording to disallow any synchronization API from being called from within `unseq / par_unseq`.

Allowing Atomic Writes and Atomic Reduction Operations in unsequenced execution increases the set of concurrent algorithms that can be implemented in the lowest-common denominator of hardware that C++ supports. In particular, many hardware architectures that can accelerate `unseq / par_unseq` but cannot accelerate `par` (e.g. most non-NVIDIA GPUs), provide acceleration for atomic reduction operations.

We propose to make lock-free atomic operations that are not reads vectorization safe to enable calling them from unsequenced execution. Atomic operations that read remain vectorization-unsafe and therefore UB:

```
for_each(par_unseq, ..., [&](...) {
    assert(atom.is_lockfree); // Only for lock-free atomics
    atom.store(42);           // OK: vectorization-safe
    atom.add(1);              // OK: vectorization-safe
    atom.fetch_add(1);        // UB: vectorization-unsafe
    atom.exchange(42);        // UB: vectorization-unsafe
    while (atom.load() < 42); // UB: vectorization-unsafe
});
```

The rest of this proposal is not blocked on this extension, which could be split into a separate proposal.

Implementation impact

There is no impact on implementations using:

- OpenMP `simd` (<https://www.openmp.org/spec-html/5.0/openmpsu42.html>) `pragma for unseq` and `par_unseq`, since OpenMP supports atomics within `simd` regions.
- `pragma clang loop` (<https://clang.llvm.org/docs/LanguageExtensions.html#extensions-for-loop-hint-optimizations>) is a hint.

Generalized Atomic Reduction Operations

The outcome `x == a + (b + c)` is not valid for the following litmus test because either the atomic reduction of thread0 happens-before that of thread 1, or vice-versa, and floating-point arithmetic is not associative:

```
// Litmus test 2:
atomic<float> x = a;
void thread0() { x.add(b, memory_order_relaxed); }
void thread1() { x.add(c, memory_order_relaxed); }
```

The value of this limitation seems small, since each execution may pick a different non-deterministic order.

The cost of this limitation is significant, since it requires implementations to perform reductions sequentially using $O(N)$ operations instead of with $O(\log(N))$ tree-reduction algorithms. On GPU architectures, performing an horizontal reduction for then issuing a single atomic operation per thread group, reduces the number of atomic operation issued by up to the size of the thread group.

We propose providing generalized atomic reduction operations, defined in an analogous way to `GENERALIZED_SUM` (see [numerics.defns] (<https://eel.is/c++draft/numerics.defns>)). The " `add` " and " `add_generalized` " operations below are different, and the latter provides implementations with the flexibility to perform a tree-reduction.

We could either only provide operations with the `GENERALIZED_...` semantics for floating-point , or provide them as separate methods:

```
template <floating-point>
class atomic {
    void add(floating-point, memory_order);
    void add_generalizedg(floating-point, memory_order);
};
```

Given the non-determinism inherent to concurrent atomic operations, the value of providing a version that differs from `GENERALIZED_..` for floating-point seems low. That is, we give the `atomic<floating-point>::add/sub/...` methods `GENERALIZED_...` semantics, and do not provide any explicit methods to pick.

The rest of this proposal is not blocked on this extension, which could be split into a separate proposal.

Memory Ordering

We choose to support `memory_order_relaxed` , `memory_order_release` , and `memory_order_seq_cst` , since we do not see any issues with doing that. If this proves to be controversial, only exposing `memory_order_relaxed` would still be valuable.

Formalization

Herd already support these for STADD on Arm, and the NVIDIA Volta Memory Model supports these for red on PTX. If we decide to pursue this exposure direction, this proposal should be blocked on extending Herd's RC11 with this extension to ensure it is sound.

Wording

Unsequenced support

Add to [intro.execution] (<https://eel.is/c++draft/basic.exec#intro.execution-10>):

Except where noted, evaluations of operands of individual operators and of subexpressions of individual expressions are unsequenced.

[Note 5: In an expression that is evaluated more than once during the execution of a program, unsequenced and indeterminately sequenced evaluations of its subexpressions need not be performed consistently in different evaluations. — end note]

The value computations of the operands of an operator are sequenced before the value computation of the result of the operator. If a side effect on a memory location ([intro.memory]) is unsequenced relative to either another side effect on the same memory location or a value computation using the value of any object in the same memory location, and they are not lock-free atomic read operations ([atomics]) or potentially concurrent ([intro.multithread]), the behavior is undefined.

[Note 6: The next subclause imposes similar, but more complex restrictions on potentially concurrent computations. — end note]

[Example 3:

```
void g(int i) {  
    i = 7, i++, i++;           // i becomes 9  
  
    i = i++ + 1;              // the value of i is incremented  
    i = i++ + i;              // undefined behavior  
    i = i + 1;                // the value of i is incremented  
}
```

— end example]

Add to [algorithms.parallel.defns] (<https://eel.is/c++draft/algorithms.parallel.defns#5>):

Note: we should consider exempting stores as well.

A standard library function is vectorization-unsafe if it is specified to synchronize with another function invocation, or another function invocation is specified to synchronize with it, and if it is not a memory allocation or deallocation function, or lock-free atomic reduction operation.

[Note 2: Implementations must ensure that internal synchronization inside standard library functions does not prevent forward progress when those functions are executed by threads of execution with weakly parallel forward progress guarantees. — end note]

[Example 2:

```
int x = 0;
std::mutex m;
void f() {
    int a[] = {1,2};
    std::for_each(std::execution::par_unseq, std::begin(a), std::end(a), [&](int i) {
        std::lock_guard<mutex> guard(m); // incorrect: lock_guard constructor call
        ++x;
    });
}
```

The above program may result in two consecutive calls to `m.lock()` on the same thread of execution (which may deadlock), because the applications of the function object are not guaranteed to run on different threads of execution. — end example]

Forward progress

Modify `intro.progress#1` (<https://eel.is/c++draft/intro.progress#1>) as follows:

Note: we should consider exempting stores as well.

The implementation may assume that any thread will eventually do one of the following:

1. terminate,
2. make a call to a library I/O function,
3. perform an access through a volatile glvalue, or
4. perform a synchronization operation or an atomic operation that is not an atomic reduction operation .

[Note 1: This is intended to allow compiler transformations such as removal of empty loops, even when termination cannot be proven. — end note]

Modify `intro.progress#3` (<https://eel.is/c++draft/intro.progress#3>) as follows:

Note: we should consider exempting stores as well.

During the execution of a thread of execution, each of the following is termed an execution step:

1. termination of the thread of execution,
2. performing an access through a volatile glvalue, or
3. completion of a call to a library I/O function, or a synchronization operation, or an atomic operation that is not an atomic reduction operation.

No acquire sequences support

Modify `[atomics.fences]` (<https://eel.is/c++draft/atomics.fences>) as follows:

33.5.11 Fences`[atomics.fences]`

1. This subclause introduces synchronization primitives called fences. Fences can have acquire semantics, release semantics, or both. A fence with acquire semantics is called an acquire fence. A fence with release semantics is called a release fence.
2. A release fence A synchronizes with an acquire fence B if there exist atomic operations X and non-reduction-atomic operation Y, both operating on some atomic object M, such that A is sequenced before X, X modifies M, Y is sequenced before B, and Y reads the value written by X or a value written by any side effect in the hypothetical release sequence X would head if it were a release operation.
3. A release fence A synchronizes with an atomic operation B that performs an acquire operation on an atomic object M if there exists an atomic operation X such that A is sequenced before X, X modifies M, and B reads the value written by X or a value written by any side effect in the hypothetical release sequence X would head if it were a release operation.
4. An atomic operation A that is a release operation on an atomic object M synchronizes with an acquire fence B if there exists some atomic operation X on M such that X is sequenced before B and reads the value written by A or a value written by any side effect in the release sequence headed by A.

Atomic Reduction Operation APIs

The following operations perform arithmetic computations. The correspondence among key, operator, and computation is specified in Table 145 (<https://eel.is/c++draft/atomics#tab:atomic.types.int.comp>):

key	op	computation
add	+	addition
sub	−	subtraction
max		maximum
min		minimum
and	&	bitwise and
or		bitwise inclusive or
xor	^	bitwise exclusive or

Add to [atomics.ref.int] (<https://eel.is/c++draft/atomics.ref.int>):

void key(integral-type operand, memory_order order = memory_order_seq_cst) const r

- Preconditions: order is memory_order relaxed, memory_order release, or memory_order_seq_cst.
- Effects: Atomically replaces the value referenced by *ptr with the result of the computation applied to the value referenced by *ptr and the given operand. Memory is affected according to the value of order. These operations are atomic read-modify-write operations ([intro.races]) and atomic reduction operations ([atomics.fences]). Lock-free atomic reduction operations are vectorization-safe ([algorithms.parallel.defns]).
- Remarks: For signed integer types, the result is as if the object value and parameters were converted to their corresponding unsigned types, the computation performed on those types, and the result converted back to the signed type.
[Note 2: There are no undefined results arising from the computation. — end note]
- Remarks: For floating point types, add and sub perform generalized [numerics.defns] arithmetic.

Analogously for all other `std::atomic` and `std::atomic_ref` types and specializations: [atomics.types.int] (<https://eel.is/c++draft/atomics.types.int>), [atomics.types.float], [atomics.types.pointer] (with `difference_type` operand), [atomics.ref.float], [atomics.ref.pointer] (with `difference_type` operand), etc.